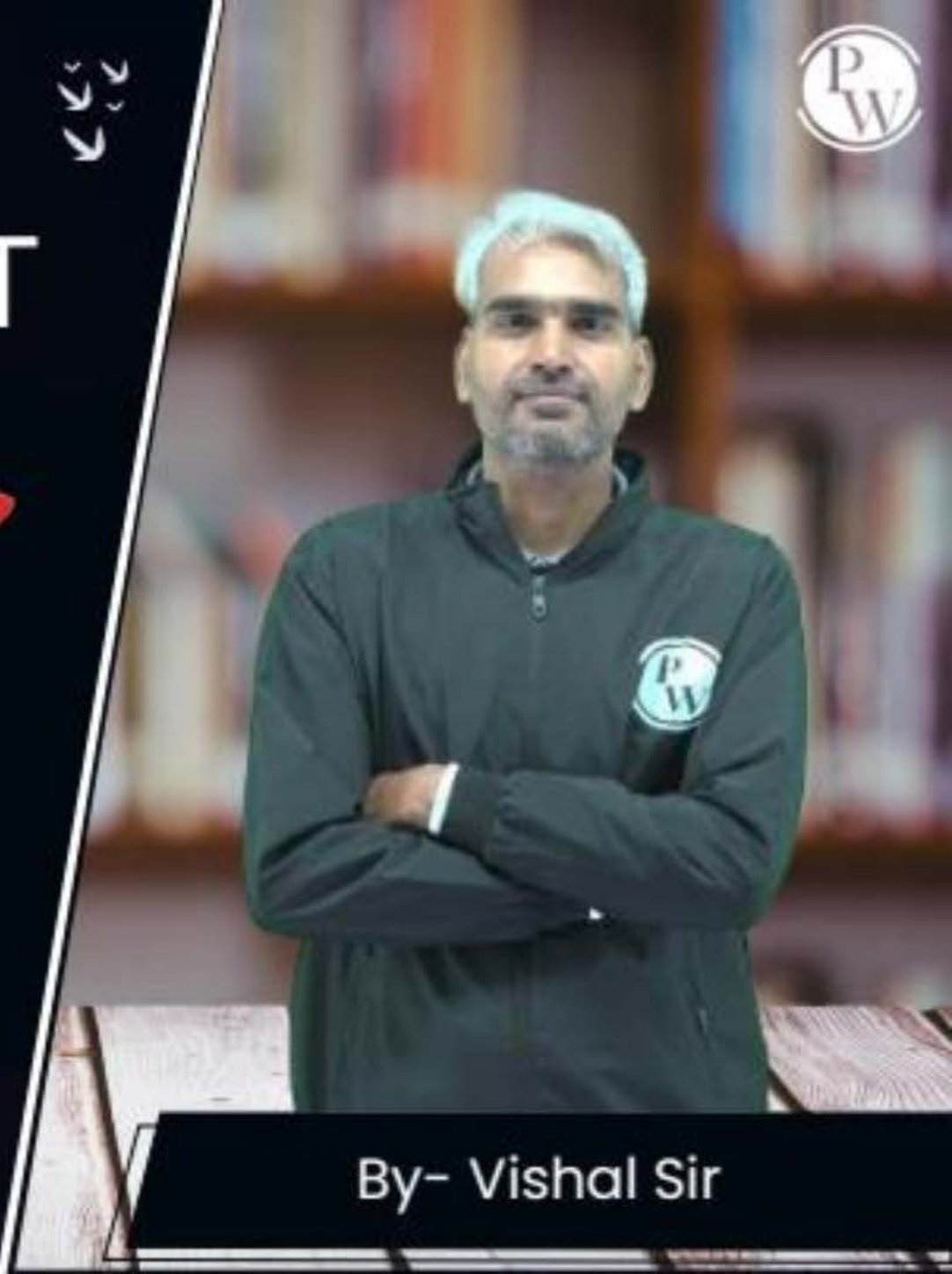
Computer Science & IT



Database Management System

B Tree and B+ Tree

Lecture No. 01



By- Vishal Sir

Recap of Previous Lecture



- **Topic** Insertion in B tree
- **Topic** Deletion from B tree



Slide

Topics to be Covered



- Topic — Deletion from B tree { Problem }
- Topic — Insertion in B+ tree { Algo + Problem }

Slide

* Insertion in B tree :→ Insertion will always be Performed at leaf node.

- Let order of a node of B tree = P {∴ Max no. of keys = P-1}

1. Search for the leaf node that can hold the key value that needs to be inserted.
2. If leaf node is not already full {i.e. number of keys < (P-1)} then simply insert the key in the ascending order
3. If leaf node is already full {i.e. no. of keys in leaf node = (P-1)} then because of insertion of this new key there will be overflow in that node. In that case we will insert the new key in the leaf node in ascending order, then we will split the node into two parts and we will promote the median position key to the parent node

③ (i) If parent node was not already full, then it will be able to accomodate the Promoted Key, and no further action will be required.

③ (ii) • If parent node was already full, then split that node into two parts, and promote the median Position Key to the parent node of that node.
* This may go upto root node, and if root node is also full, then we will split the root node as well into two parts, And the median Position Key will become the new root node {i.e: Height of tree will increase by 1}

Q:- Let order of B tree = 4.
And keys to be inserted are.
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25
in the same order.

* Construct the B tree

* Order of a node of B tree = 4.
  * Maximum number of child pointer a node can have = 4
  * Maximum number of keys a node of B tree can have = 4-1 = 3

⇒ Min no. of keys necessary in a non-root node = $\lceil \frac{p}{2} \rceil - 1$
$= \lceil \frac{4}{2} \rceil - 1 = 1$

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25

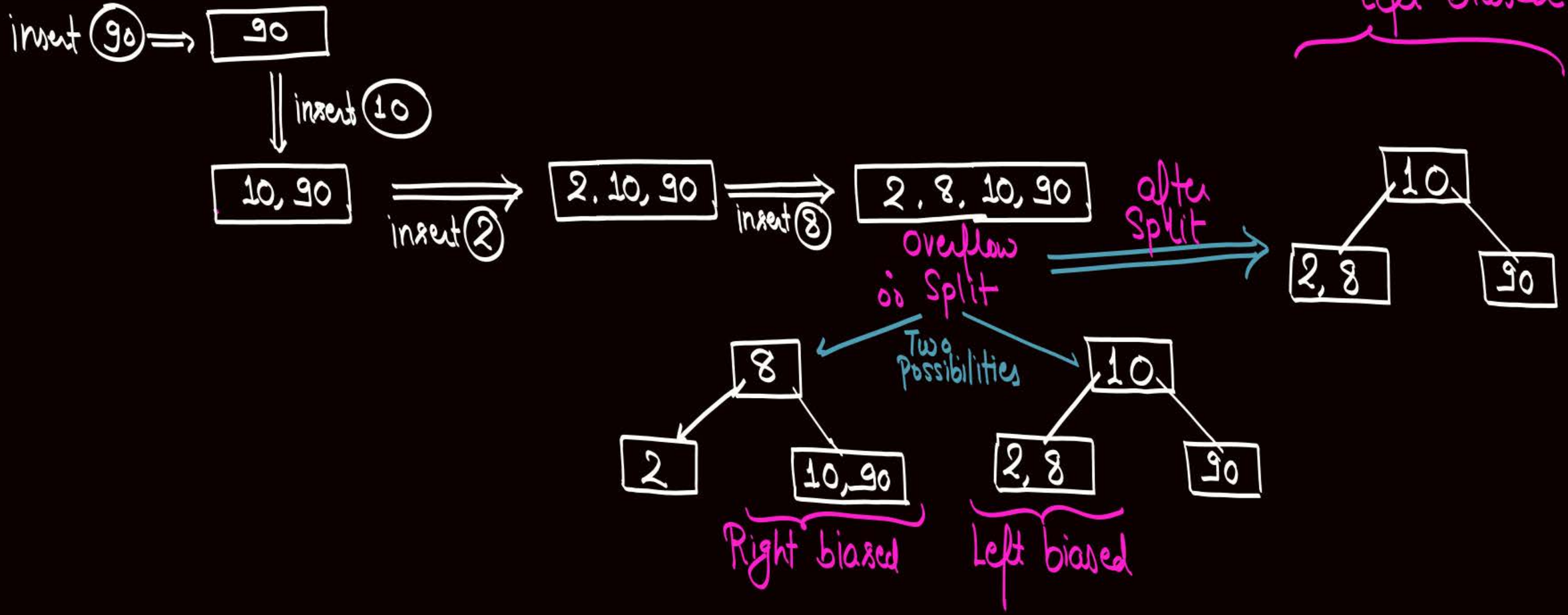
Not required

⇒ Min no. of keys necessary in a non-root node = $\lceil \frac{p}{2} \rceil - 1$
$= \lceil \frac{4}{2} \rceil - 1 = 1$

* Order of a node of B tree = 4.
  * Maximum number of child pointer a node can have = 4
  * Maximum number of keys a node of B tree can have = 4-1 = 3

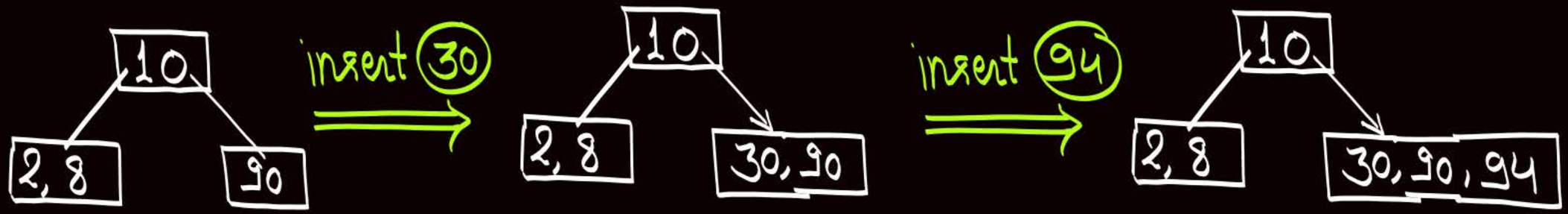
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25

Let us choose left biased



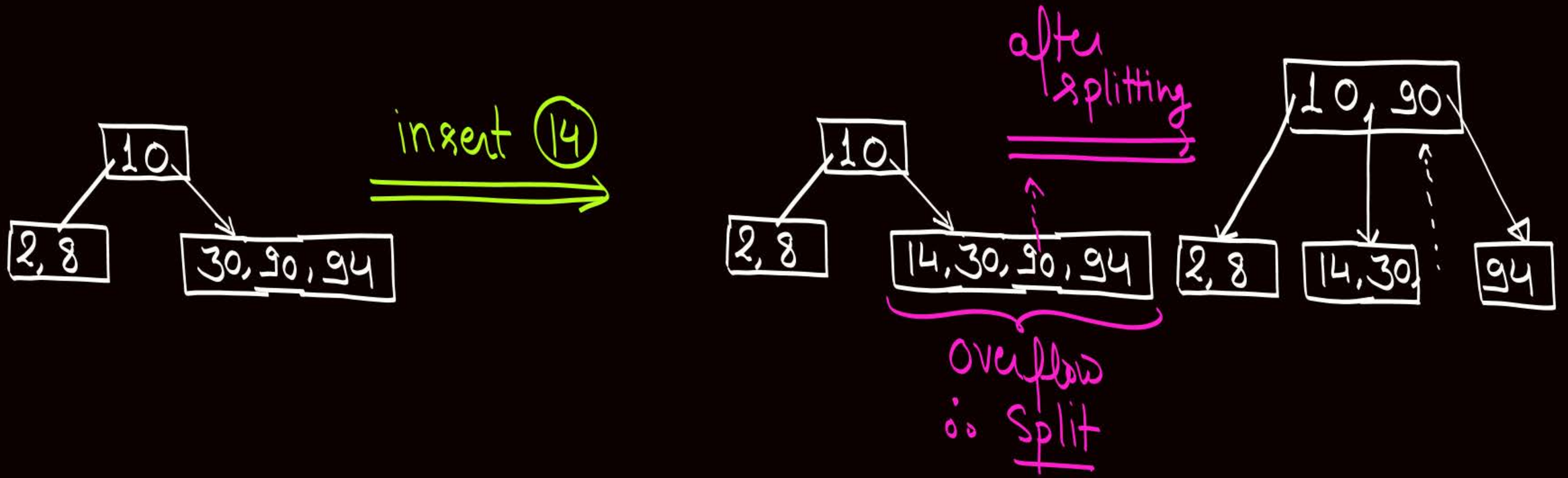
* Order of a node of B tree = 4.
  * Maximum number of child pointer a node can have = 4
  * Maximum number of keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



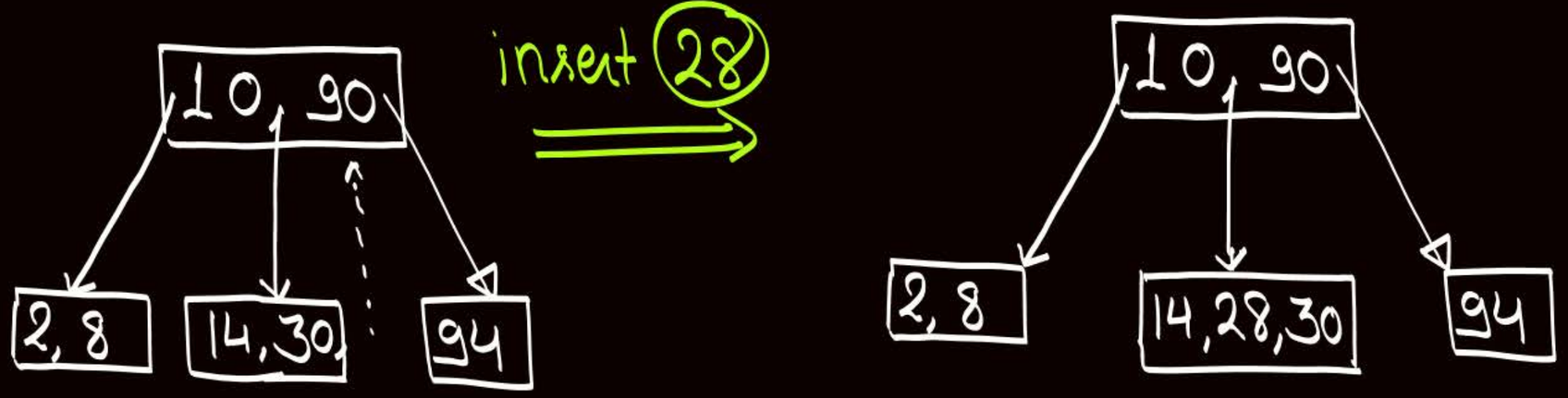
* Order of a node of B tree = 4.
  * Maximum number of child pointer a node can have = 4
  * Maximum number of keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



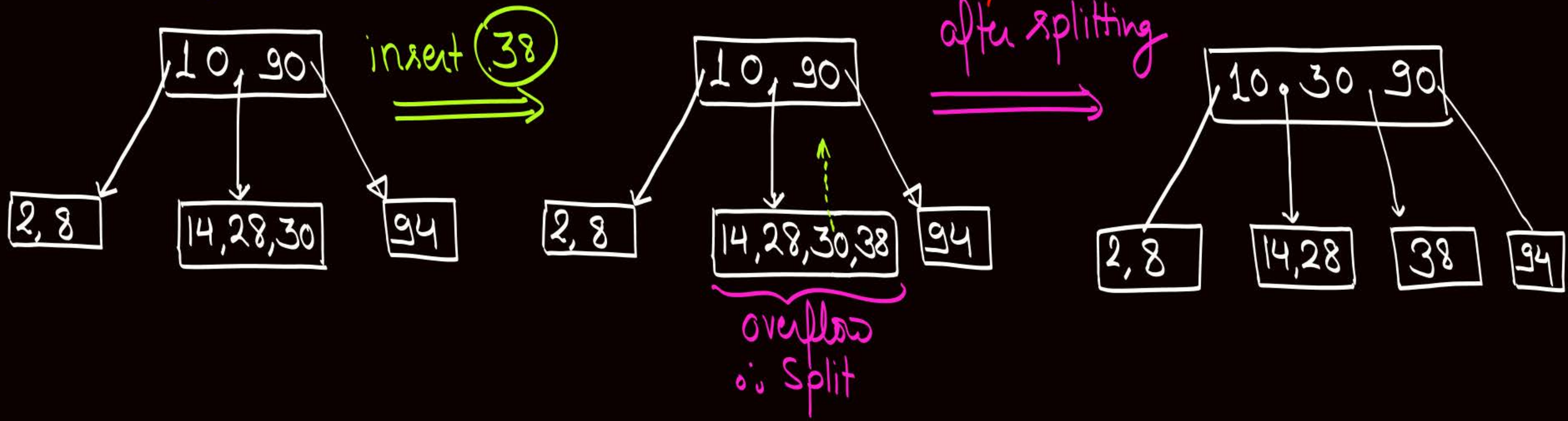
* Order of a node of B tree = 4.
  * Maximum number of child pointer a node can have = 4
  * Maximum number of keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



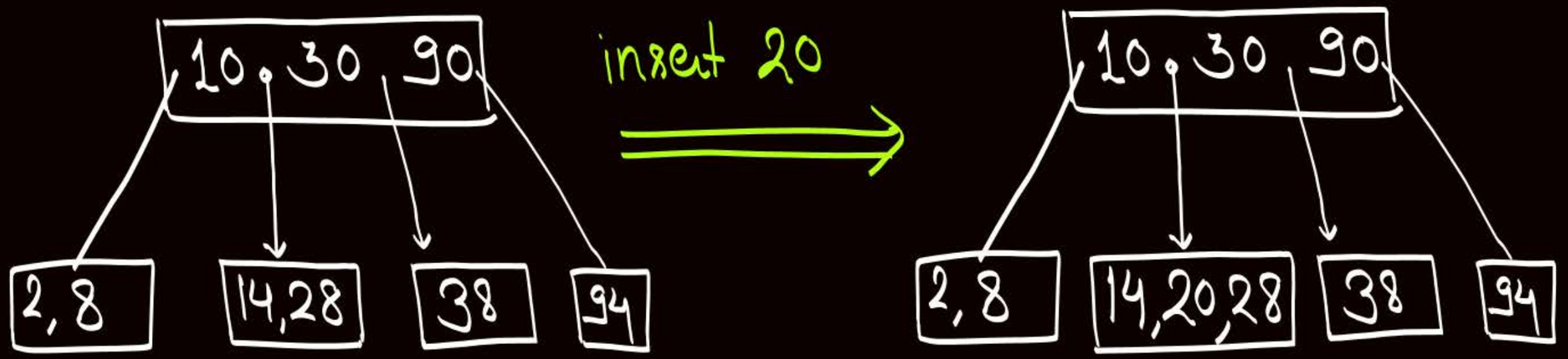
* Order of a node of B tree = 4.
  * Maximum number of child pointer a node can have = 4
  * Maximum number of keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



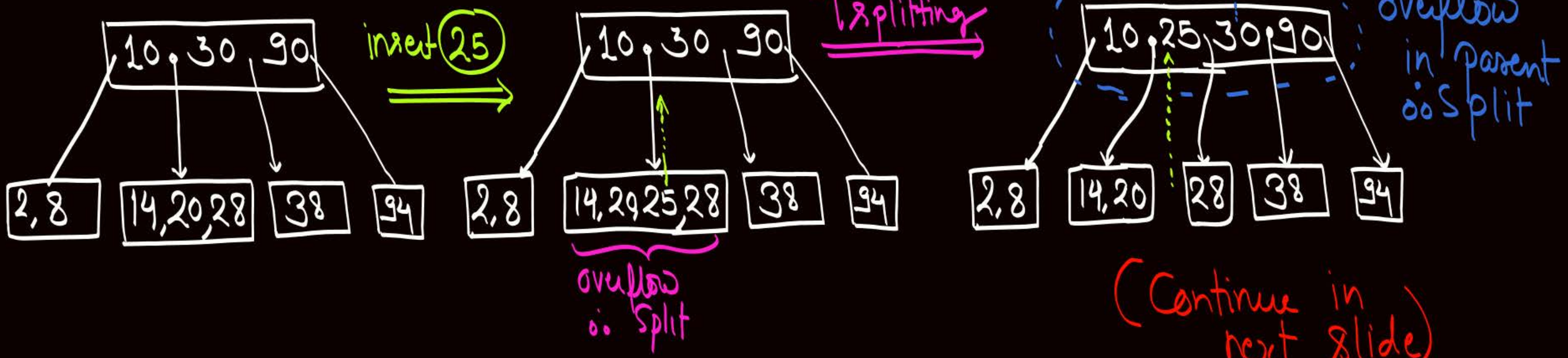
* Order of a node of B tree = 4.
  * Maximum number of child pointer a node can have = 4
  * Maximum number of keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



* Order of a node of B tree = 4.
  * Maximum number of child Pointer a node can have = 4
  * Maximum number of Keys a node of B tree can have = 4-1 = 3

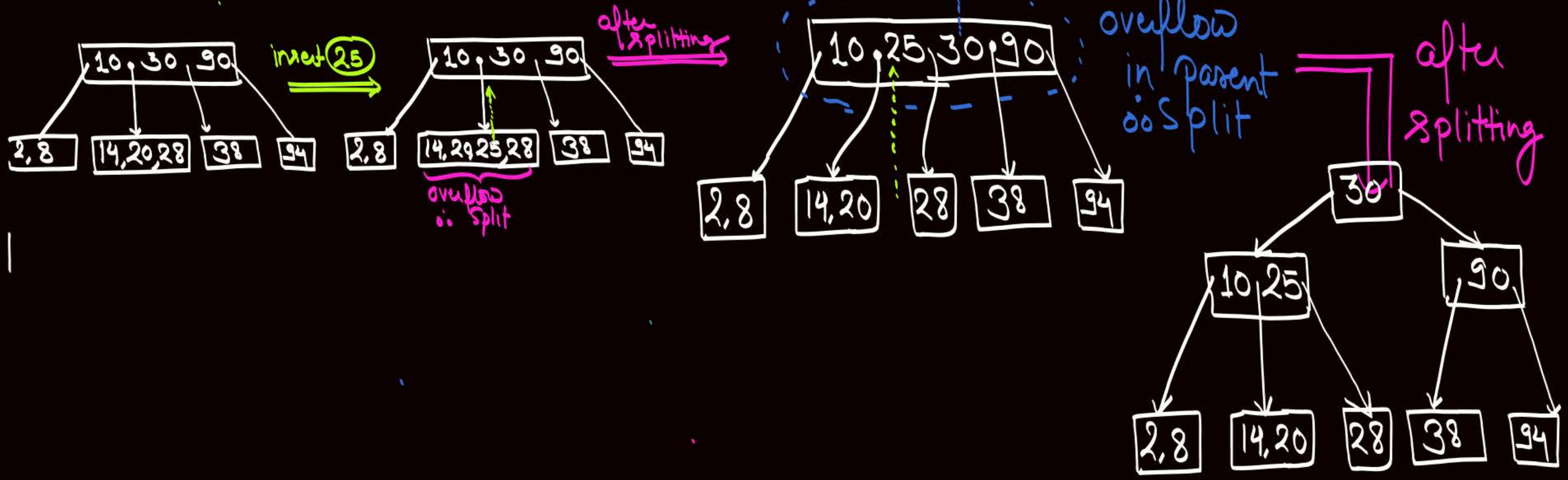
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



(Continue in next slide)

* Order of a node of B tree = 4.
  * Maximum number of child Pointer a node can have = 4
  * Maximum number of Keys a node of B tree can have = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



Deletion from B tree

* Deletion from B Tree :-

→ { "key" that needs to be deleted may be in a leaf node or may be in an internal node }

① If key that needs to be deleted is present in internal node of B tree, then replace it by
- ✓ in-order predecessor {i.e. largest key in left sub-tree}
  or
- ✓ in-order successor {i.e smallest key from right sub-tree}

and then delete the key from leaf node

② If key is already at the leaf node delete it from that leaf node

* ① After deletion of the key from leaf node if leaf node still contain at least the minimumum numer of keys required, then no further action is needed

② After deletion of the key from leaf node if leaf node becomes deficient {i.e. Contains less than the minimum number of keys required}

2(i) Check with your immediate sibling if they can help.

{first check with left sibling, then check with right sibling}

ⓐ If left sibling can help then borrow (re-distribute) from left sibling
Largest key from the left sibling will be promoted to Parent node, and corresponding key from parent {i.e. key b/w this two nodes} will be transferred to deficient node

ⓑ If right sibling can help then borrow (re-distribute) from right sibling
Smallest key from right sibling will be promoted to Parent node, and corresponding key from Parent node {i.e; key between those two nodes} will be transferred to deficient node

2 (ii) If none of your immediate sibling can help, then

- merge deficient node with one of its sibling along with one key (i.e. the key b/w deficient node & its sibling) from its parent node into a single node., as a result of this process the parent node may become deficient

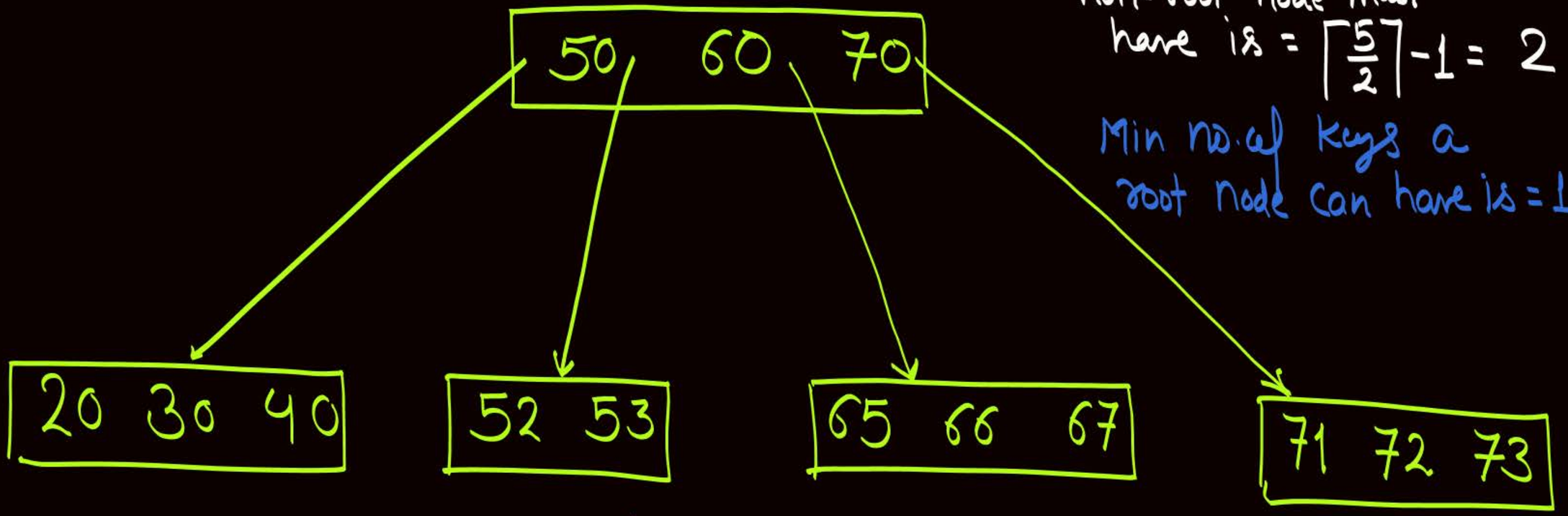
* If parent node does not become deficient then stop.
* If parent node become deficient, then the same process will be repeated (i) Ask from their sibling if they can help.
  (ii) If not, then merge with one sibling along with one key from parent node
  - This may go upto root node, if root node becomes deficient then height of tree will reduce by "1".

Q:- Consider the following B tree of order = 5.

↓

Min. no. of keys a non-root node must have is = $\lceil \frac{5}{2} \rceil - 1 = 2$

Min no. of keys a root node can have is = 1



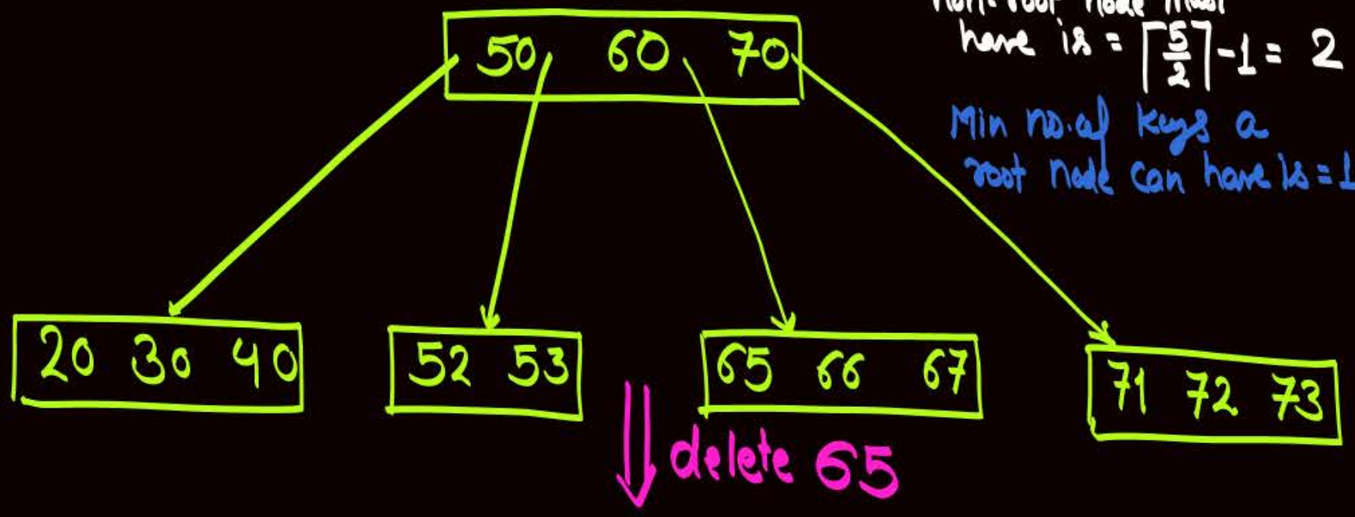
Delete the following keys from the above B tree in the same order

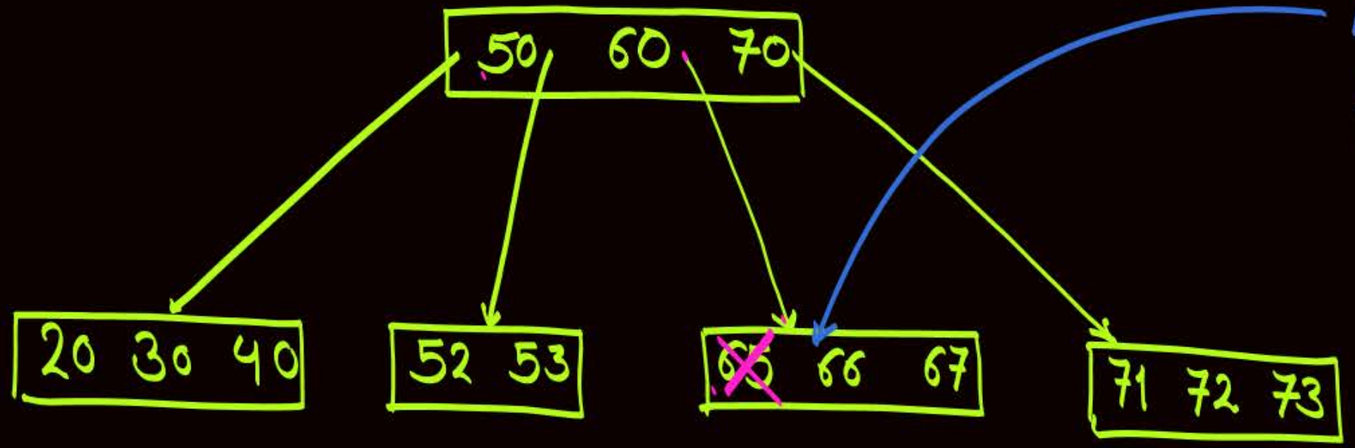
65, 53, 66, 52, 60

Q:- Consider the following B tree of order = 5.

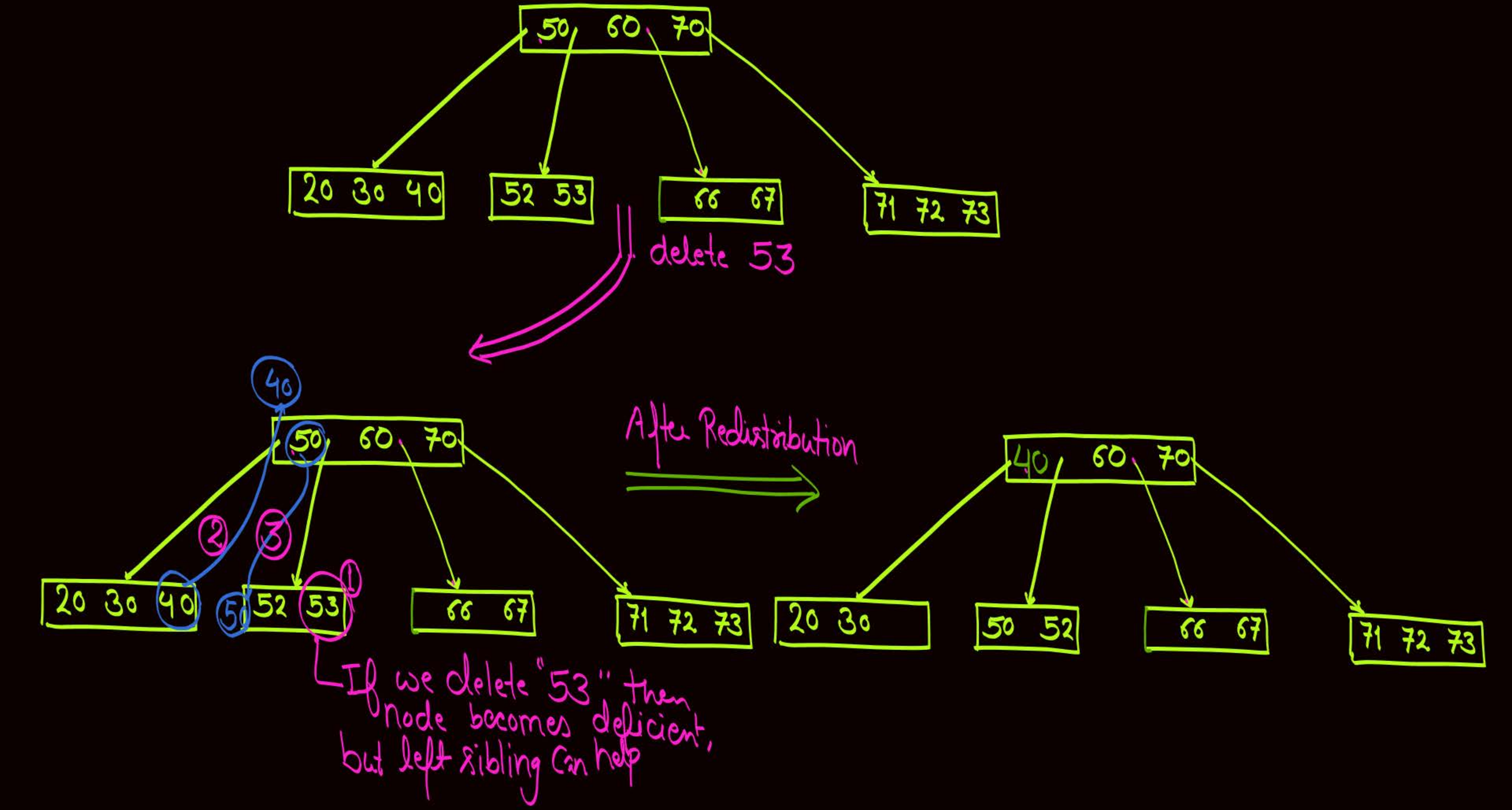
Min. no. of keys a non-root node must have is = $\lceil \frac{5}{2} \rceil - 1 = 2$

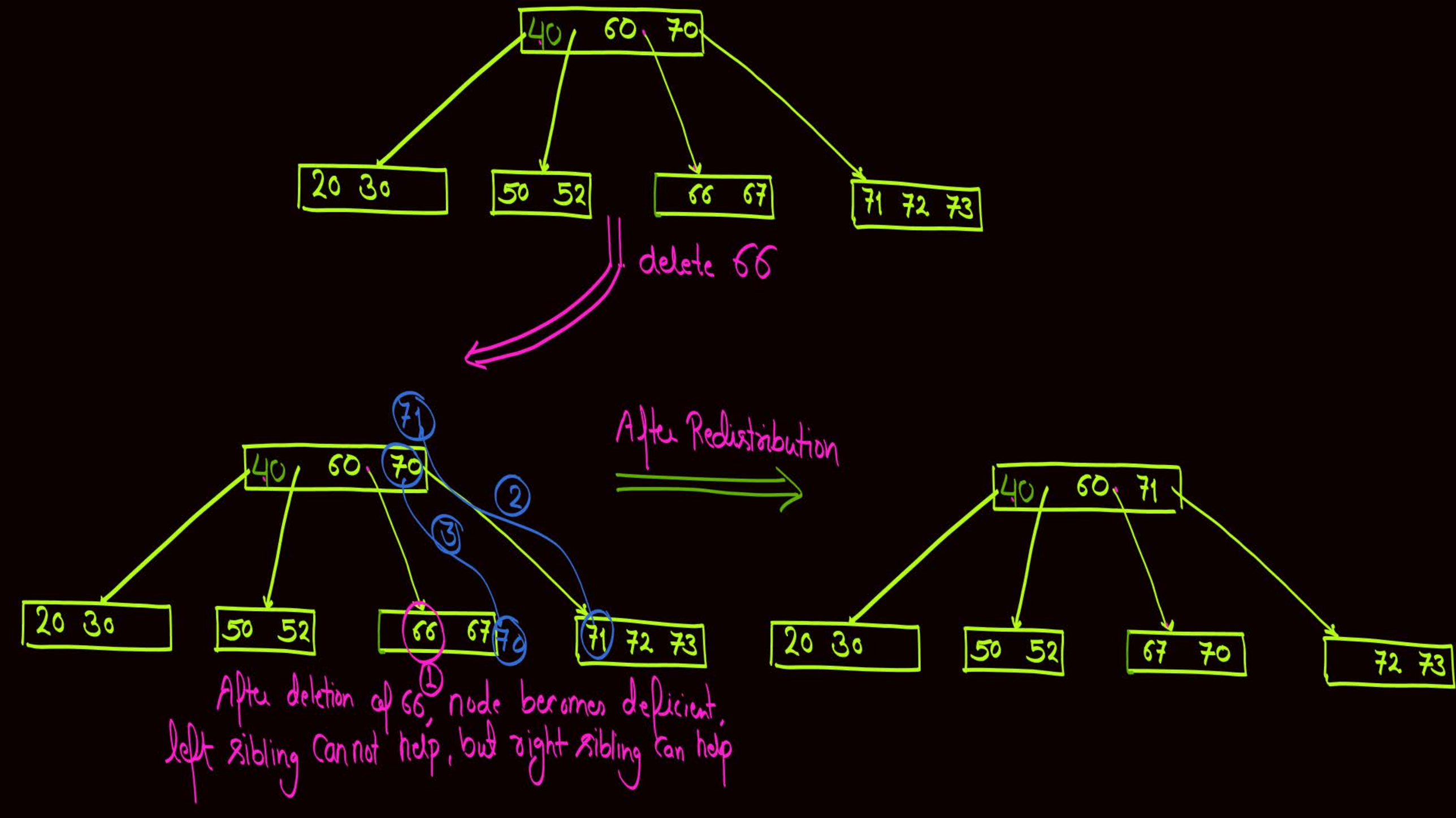
Min no. of keys a root node can have is = 1

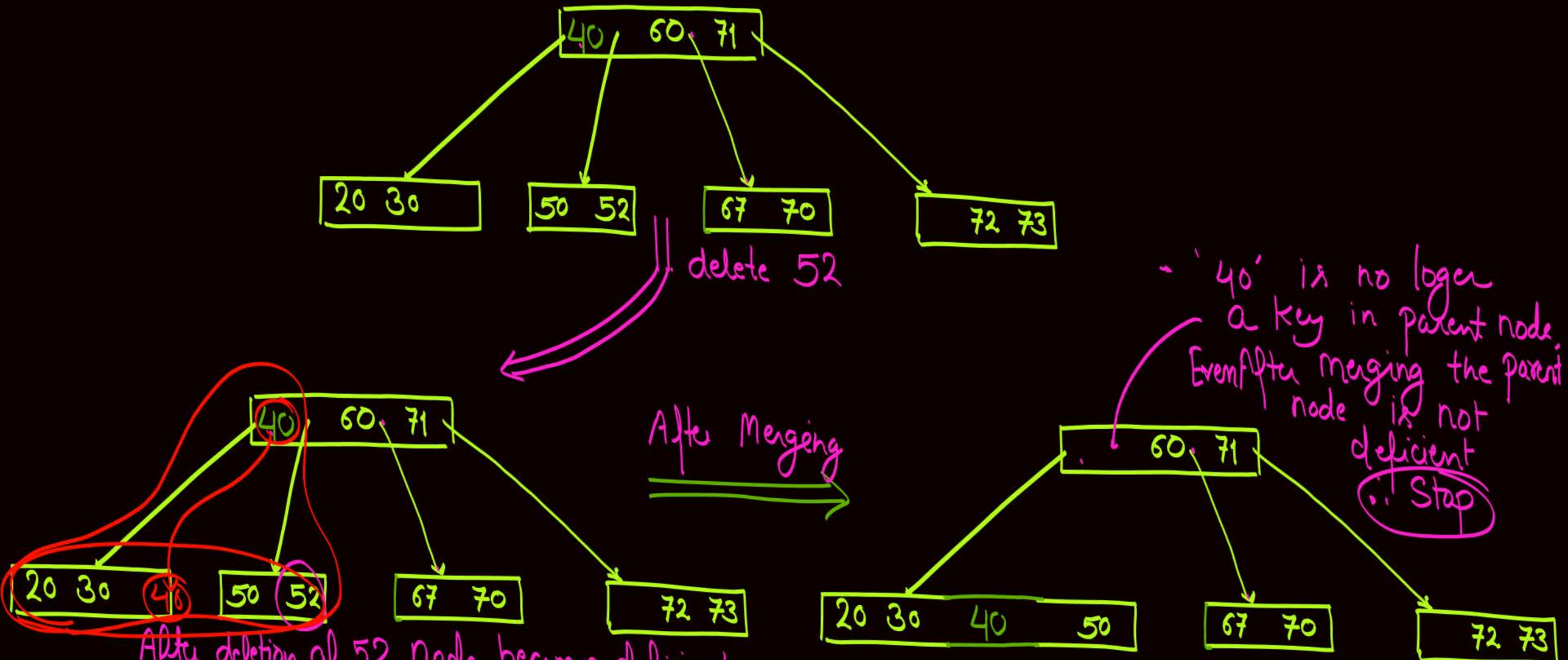


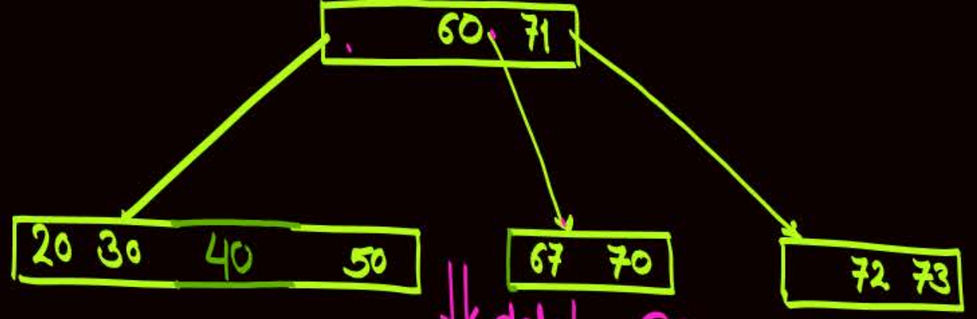


After deletion of 65 there is no underflow i.e No further action is required


50 60 70
20 30 40
52 53
66 67
71 72 73
delete 53
40
50 60 70
2
3
20 30 40
5
52 53
1
66 67
71 72 73
If we delete "53" then
node becomes deficient,
but left sibling can help
After Redistribution
40 60 70
20 30
50 52
66 67
71 72 73

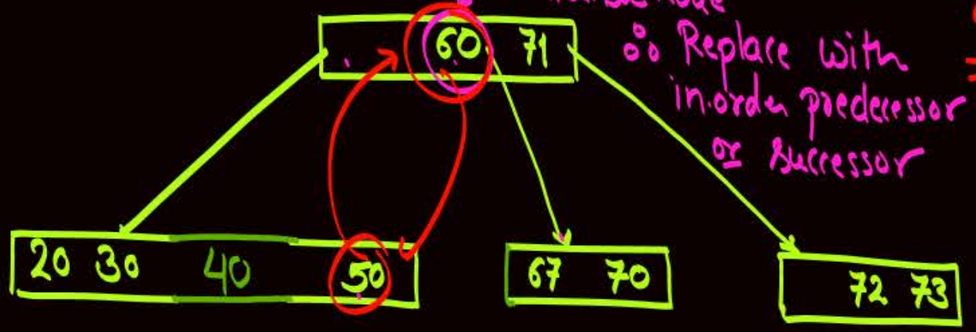

40 60 70
20 30
50 52
66 67
71 72 73
delete 66
71
40 60 70
2
3
20 30
50 52
66 67 70
71 72 73
1
After deletion of 66, node becomes deficient.
left sibling cannot help, but right sibling can help
After Redistribution
40 60 71
20 30
50 52
67 70
72 73


40 60 71
20 30
50 52
67 70
72 73
delete 52
40 60 71
20 30 40
50 52
67 70
72 73
After Merging
60 71
20 30 40 50
67 70
72 73
'40' is no longer a key in parent node. Even after merging the parent node is not deficient
∴ Stop
After deletion of 52, node becomes deficient & None of the sibling can help.
∴ Merge with one sibling along with one key from parent node

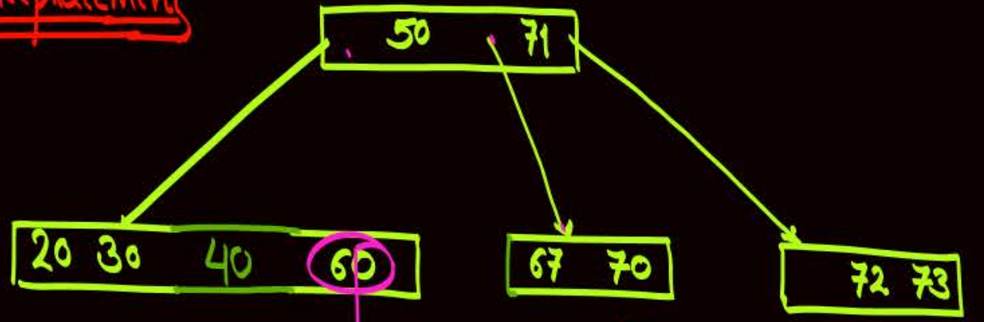


⇓ delete 60

'60' is in internal node
∴ Replace with in-order predecessor or successor

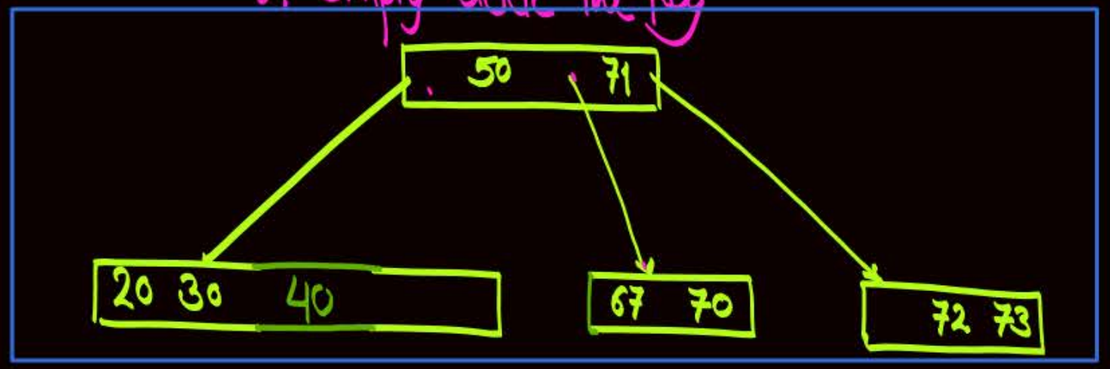


after Replacement



Now delete 60 from leaf node
After deletion there is no under-flow
∴ Simply delete the key

Final tree, after all deletions from given B tree ⟹



Insertion in to B+ tree

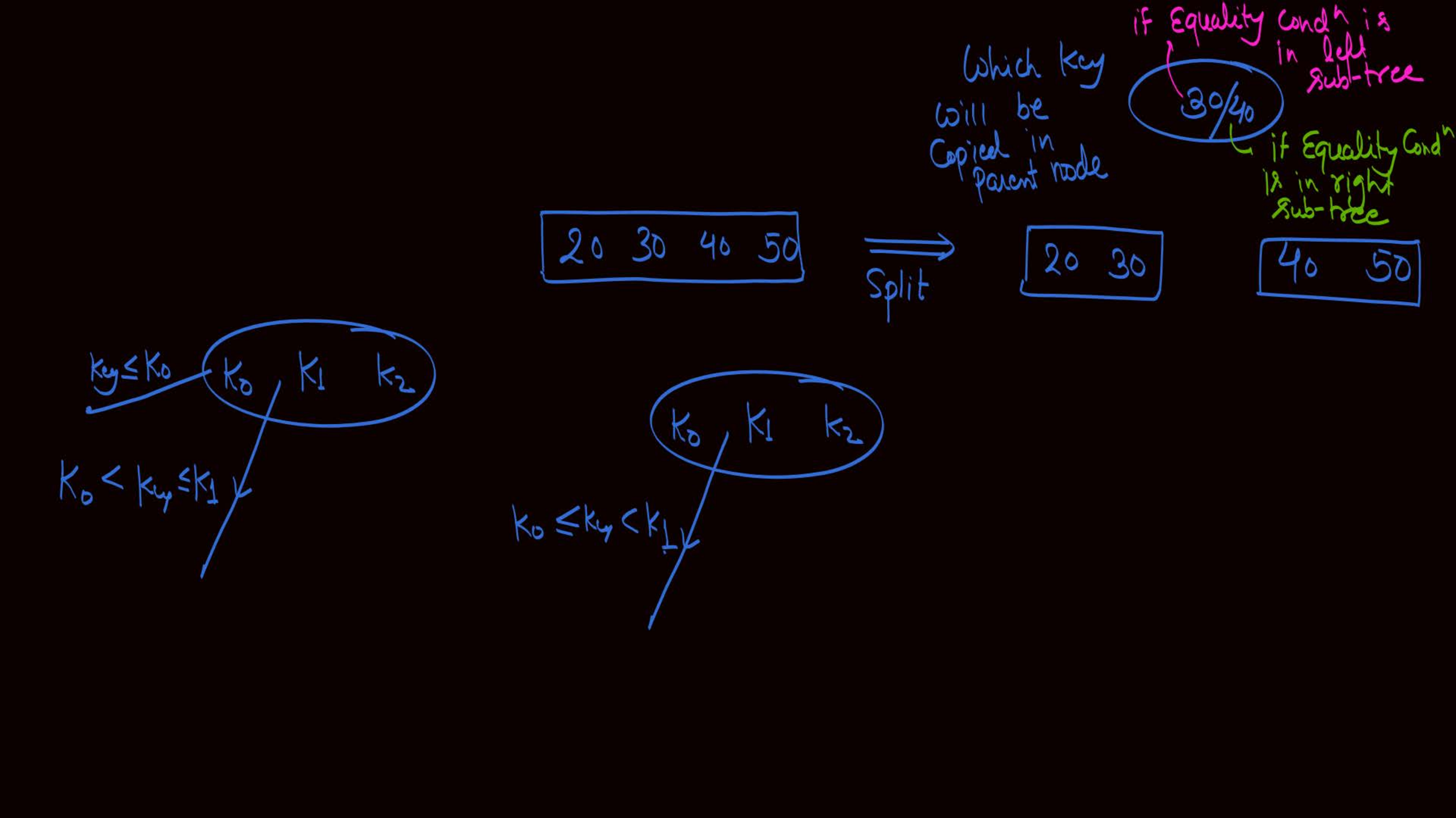


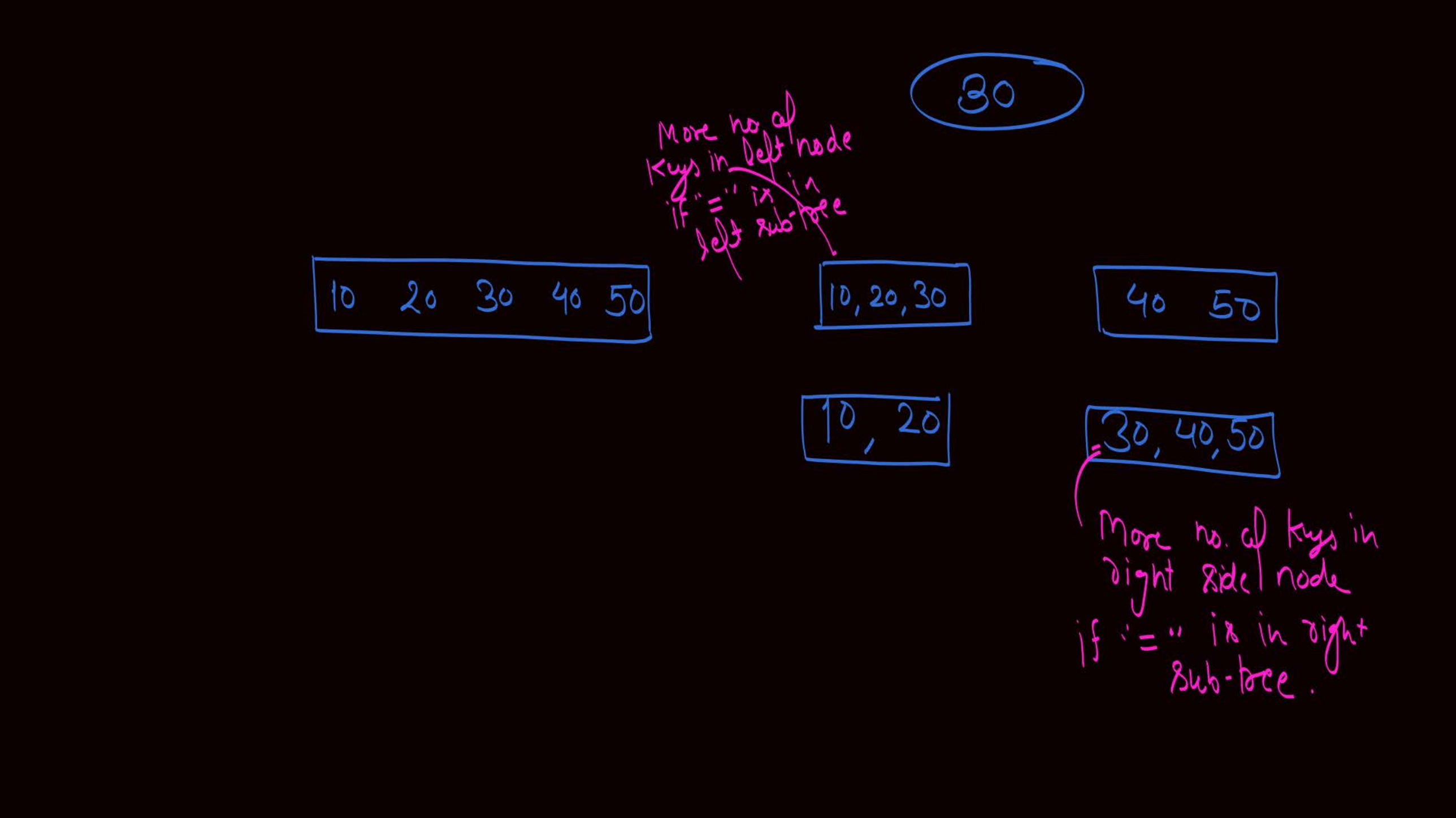
Topic : Insertion in B+ tree

- Insertion will be performed at leaf node, therefore search for appropriate leaf node
1. If leaf node is not full then insert the key into the leaf node in ascending order.
2. If leaf node is full,
   a) Insert the key in the ascending order.
   b) Split the node into two parts, and copy the median position key into the parent node, Because of this if there is overflow in the parent node then split the parent node as well.

because each key must be present in leaf node
∴ We do not promote it to parent node

Note:- While splitting the internal node, the median position key will be promoted (Not Copied) to its parent node


Which key will be Copied in parent node
if Equality Cond^n is in left sub-tree
30/40
if Equality Cond^n is in right sub-tree
20 30 40 50
Split
20 30
40 50
Key ≤ K0
K0 K1 K2
K0 < Key ≤ K1
K0 K1 K2
K0 ≤ Key < K1


30
More no of keys in left node if "=" is in left sub-tree
10 20 30 40 50
10, 20, 30
40 50
10, 20
30, 40, 50
More no. of keys in right side node if "=" is in right sub-tree.

#Q. Let,

- Order of internal node of B+ tree = 3, and
- Order of leaf node of B+ tree = 2

Insert 5, 15, 25, 35, 45 into the B+ tree in the same order

Order of internal node = 3 →

Max no. of keys in internal node = $3 - 1 = 2$

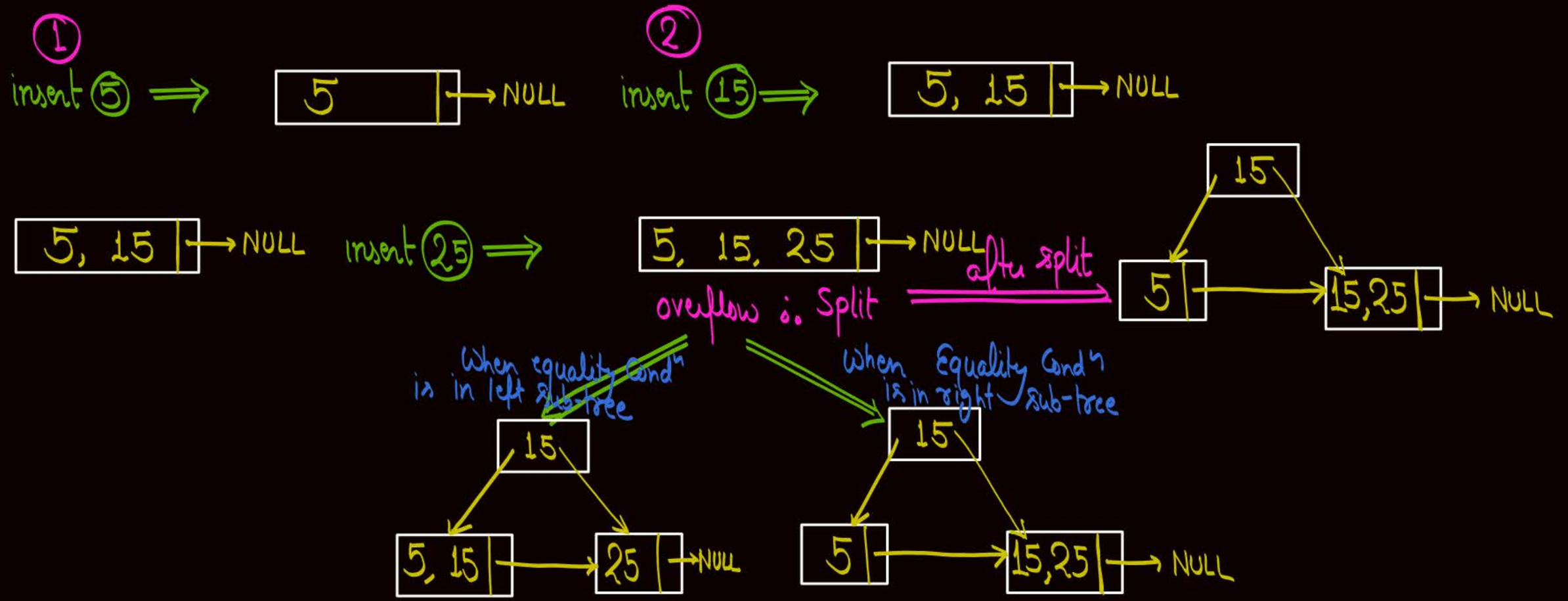
Min no. of keys required in internal node = $\lceil \frac{3}{2} \rceil - 1 = 2 - 1 = 1$

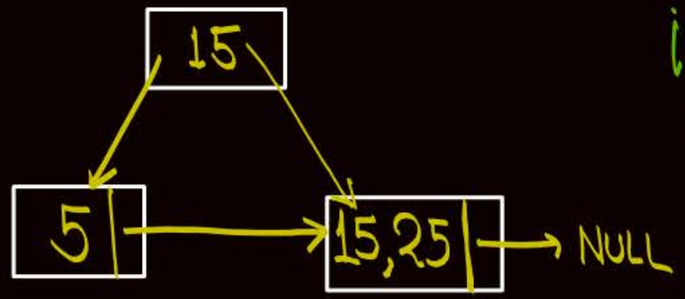
Order of leaf node = 2 →

Max no. of keys possible in leaf node = order of leaf node = 2

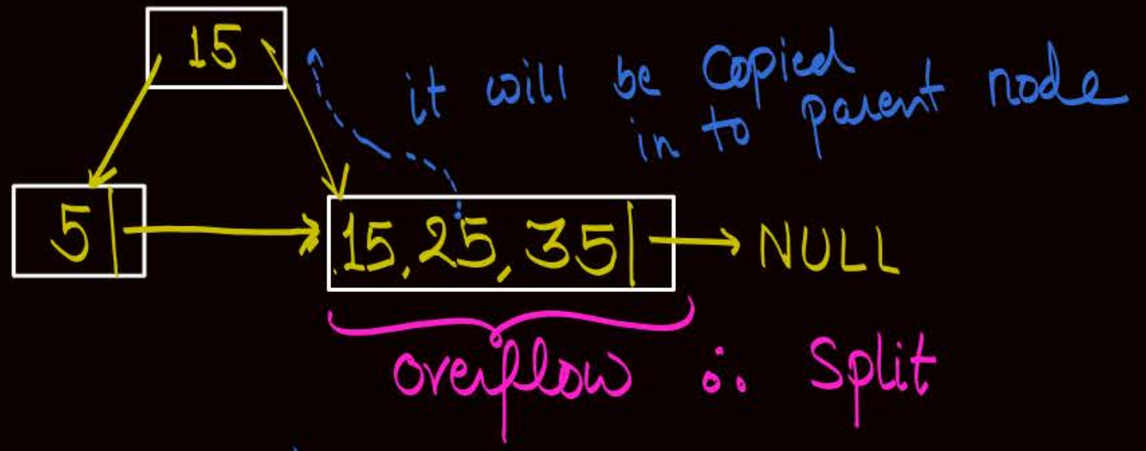
Min no. of keys required in leaf node = $\left\lceil \frac{\text{order of leaf node}}{2} \right\rceil = \lceil \frac{2}{2} \rceil = 1$

→ Min no. of keys in root node = 1

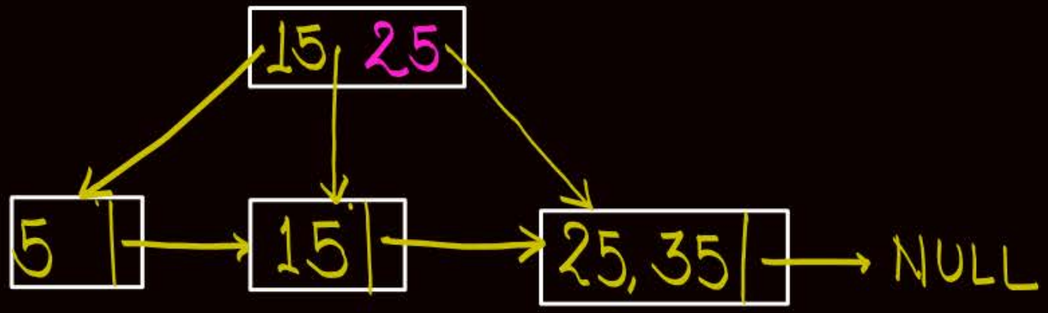

1
insert 5
5
NULL
2
insert 15
5, 15
NULL
5, 15
NULL
insert 25
5, 15, 25
NULL
overflow i. Split
after split
15
5
15,25
NULL
When equality cond^n is in left sub-tree
15
5, 15
25
NULL
When Equality Cond^n is in right sub-tree
15
5
15,25
NULL

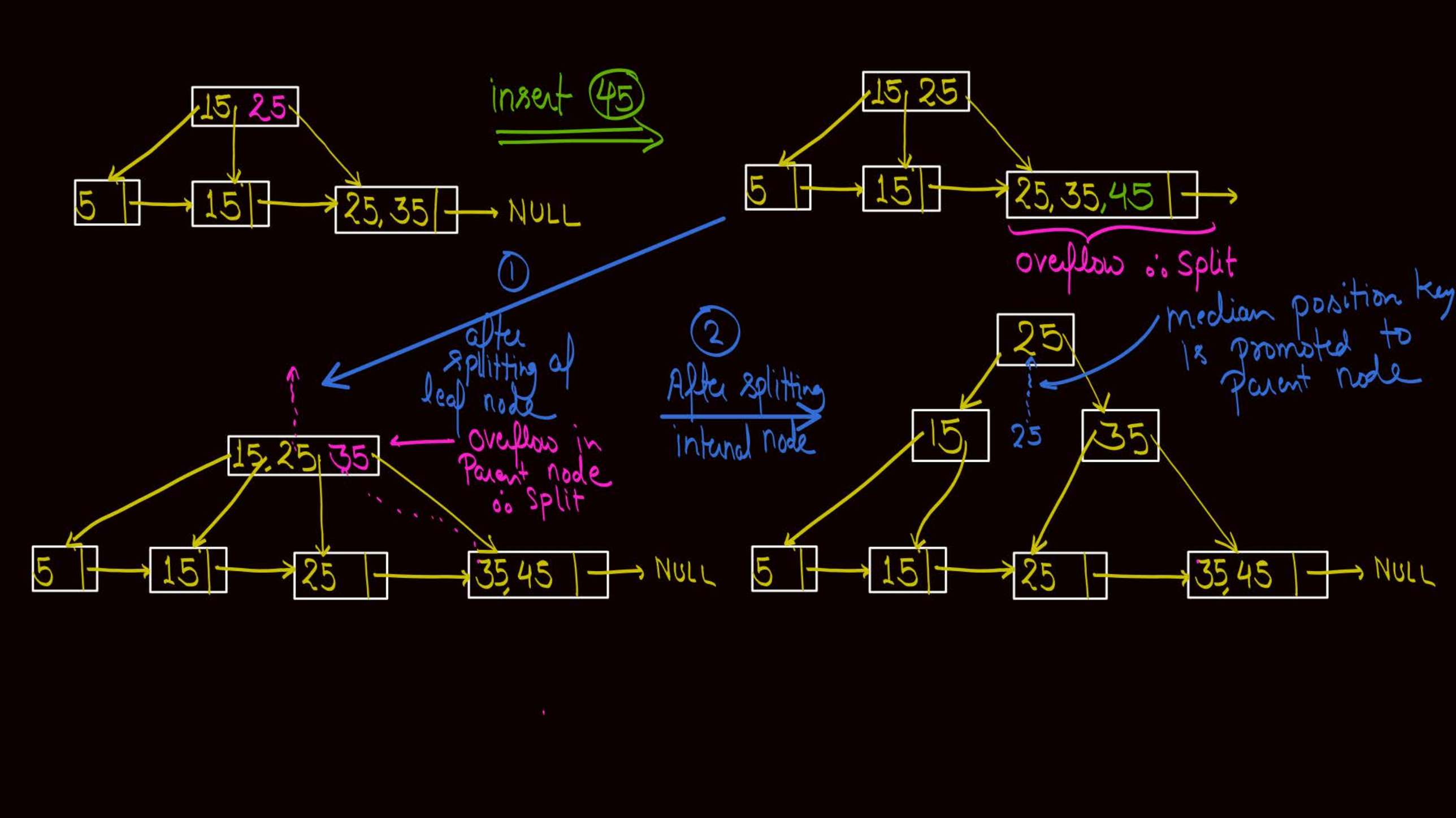


insert (35)



↓ after split




15, 25
5
15
25, 35
NULL
insert 45
15, 25
5
15
25, 35, 45
overflow ∴ Split
1
after splitting of leaf node
overflow in Parent node ∴ Split
15, 25, 35
5
15
25
35, 45
NULL
2
After splitting internal node
median position key is promoted to Parent node
25
15
25
35
5
15
25
35, 45
NULL



2 mins Summary

Topic Deletion from B tree

Topic Insertion in B+ tree

Slide

THANK - YOU